文档

1. authorization code 授权流程：https://gitlab.0voice.com/help/api/oauth2.md#authorization-code-flow

Authorization code flow

NOTE: Check the RFC spec for a detailed flow description.

The authorization code flow is essentially the same as authorization code flow with PKCE,

Before starting the flow, generate the `STATE`. It is a value that can't be predicted used by the client to maintain state between the request and callback. It should also be used as a CSRF token.

1. Request authorization code. To do that, you should redirect the user to the `/oauth/authorize` page with the following query parameters:

   ```
   https://gitlab.example.com/oauth/authorize?client_id=APP_ID&redirect_uri=REDIRECT_URI&response_type=code&state=STATE&
   ```

   This page asks the user to approve the request from the app to access their account based on the scopes specified in `REQUESTED_SCOPES`. The user is then redirected back to the specified `REDIRECT_URI`. The scope parameter is a space-separated list of scopes associated with the user. For example, `scope=read_user+profile` requests the `read_user` and `profile` scopes. The redirect includes the authorization `code`, for example:

   ```
   https://example.com/oauth/redirect?code=1234567890&state=STATE
   ```

2. With the authorization `code` returned from the previous request (shown as `RETURNED_CODE` in the following example), you can request an `access_token`, with any HTTP client. The following example uses Ruby's `rest-client`:

   ```
   parameters = 'client_id=APP_ID&client_secret=APP_SECRET&code=RETURNED_CODE&grant_type=authorization_code&redirect_uri
   RestClient.post 'https://gitlab.example.com/oauth/token', parameters
   ```

   Example response:

   ```
   {
    "access_token": "de6780bc506a0446309bd9362820ba8aed28aa506c71eedbe1c5c4f9dd350e54",
    "token_type": "bearer",
    "expires_in": 7200,
    "refresh_token": "8257e65c97202ed1726cf9571600918f3bffb2544b26e00a61df9897668c33a1",
    "created_at": 1607635748
   }
   ```

3. To retrieve a new `access_token`, use the `refresh_token` parameter. Refresh tokens may be used even after the `access_token` itself expires. This request:

   - Invalidates the existing `access_token` and `refresh_token`.
   - Sends new tokens in the response.

   ```
   parameters = 'client_id=APP_ID&client_secret=APP_SECRET&refresh_token=REFRESH_TOKEN&grant_type=refresh_token&redire
   RestClient.post 'https://gitlab.example.com/oauth/token', parameters
   ```

   Example response:

   ```
   {
     "access_token": "c97d1fe52119f38c7f67f0a14db68d60caa35ddc86fd12401718b649dcfa9c68",
     "token_type": "bearer",
     "expires_in": 7200,
     "refresh_token": "803c1fd487fec35562c205dac93e9d8e08f9d3652a24079d704df3039df1158f",
     "created_at": 1628711391
   }
   ```

NOTE: The `redirect_uri` must match the `redirect_uri` used in the original authorization request.

You can now make requests to the API with the access token returned.

Access GitLab API with `access token`

The `access token` allows you to make requests to the API on behalf of a user. You can pass the token either as GET parameter:

```
GET https://gitlab.example.com/api/v4/user?access_token=OAUTH-TOKEN
```

or you can put the token to the Authorization header:

```
curl --header "Authorization: Bearer OAUTH-TOKEN" "https://gitlab.example.com/api/v4/user"
```